

CS306 Project Phase 4

Web Integrations of SQL and NoSQL Databases

Contents

1 Introduction.....	2
2 System Design and Features	2
2.1 User Interface	2
2.2 Support Ticket System (User Side)	4
2.3 Admin Interface	6
3 Implementation Requirements	8
3.1 Project Structure	8
3.2 Trigger and Stored Procedure Responsibilities.....	8
3.3 MySQL Integration.....	8
3.4 MongoDB Integration for Support Tickets	9
3.5 Focus on Functionality	12
3.6 Resources and References	12
4 Submission and Evaluation	13
4.1 Contents of the Submission	13
4.2 Submission and Grading Policy	13
4.3 Demo Video Requirements	14

1. Introduction

This project focuses on extending our previously developed database system into a fully functional web application, supporting both user and administrator interfaces. Using **PHP** and **XAMPP** as our development stack, the goal is to integrate core database logic specifically, triggers and stored procedures with a dynamic and interactive website. In addition, we introduce a support ticket system that allows users to request assistance and administrators to manage and respond to those requests.

The application is divided into two primary components:

User Side: A homepage where users can access features that interact with database triggers and stored procedures. It also includes a support section where users can open tickets for help.

Admin Side: A dedicated interface for administrators to view and manage active support tickets submitted by users. Admins can comment on tickets and mark them as resolved.

The web application is hosted locally using XAMPP, and PHP is used to handle both MySQL and MongoDB database interactions. The triggers and stored procedures used in this project were designed and implemented in earlier phases, and this phase focuses on making them accessible and testable via the web interface. Meanwhile, the support ticket system is backed by **MongoDB**.

2. System Design and Features

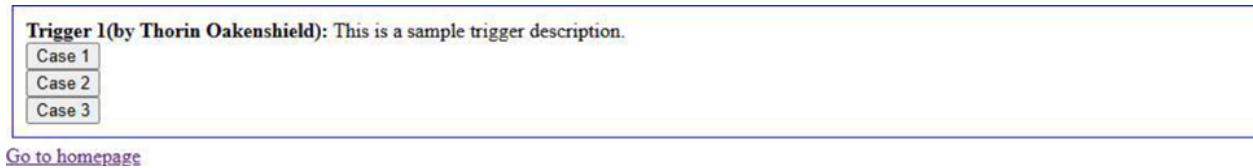
This section describes the key components of the web application, detailing how users and administrators interact with the system. Each part of the interface is designed to connect with the backend logic and database operations.

2.1. User Interface

The user side of the system is accessible via the **localhost/user URL**. This entry point leads to a homepage where users can navigate to triggers, stored procedures, or the support section. Each of these features is accessible through clearly labeled hyperlinks.

Trigger Integration

Each group member is responsible for integrating one database trigger with the web interface. The homepage displays a list of all triggers, with a brief description and the name of the person responsible for that integration. Clicking on a trigger link opens a dedicated page

A screenshot of a web page for a trigger. At the top, it says "Trigger 1(by Thorin Oakenshield): This is a sample trigger description." Below this, there are three buttons labeled "Case 1", "Case 2", and "Case 3" stacked vertically. At the bottom left, there is a link that says "Go to homepage".

Trigger 1(by Thorin Oakenshield): This is a sample trigger description.

Case 1

Case 2

Case 3

[Go to homepage](#)

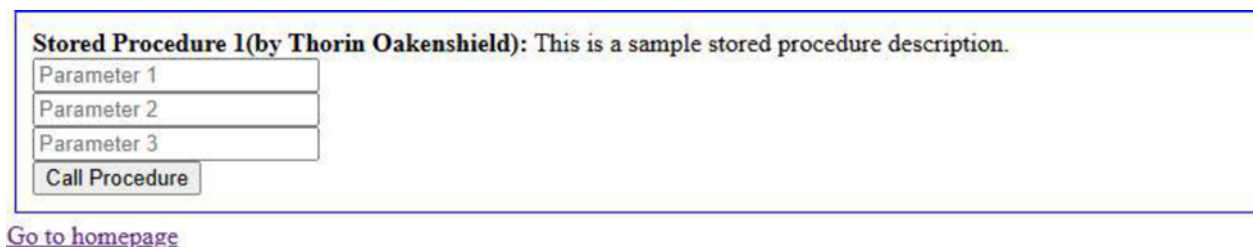
Figure 1: An example Trigger Page

describing how it works and showing interactive buttons that simulate different conditions under which the trigger would execute. After a button is clicked, the output message should be displayed within the page. Figure 1 shows an example page dedicated to a trigger.

As an example, a trigger that behaves differently depending on two input conditions will have two buttons on its page. Each button initiates a case that tests a specific behavior of the trigger through database operations. At the bottom of the page, there is a link to return to the homepage.

Stored Procedure Integration

Similarly, each member integrates a stored procedure from the previous phase. The home-page displays these procedures along with descriptions and responsible group members. Clicking on a procedure link takes the user to a form where they can provide input value use for the procedure's parameters. Figure 2 shows an example page dedicated to a stored procedure.

A screenshot of a web page for a stored procedure. At the top, it says "Stored Procedure 1(by Thorin Oakenshield): This is a sample stored procedure description." Below this, there are three input fields labeled "Parameter 1", "Parameter 2", and "Parameter 3" stacked vertically. Below the input fields is a button labeled "Call Procedure". At the bottom left, there is a link that says "Go to homepage".

Stored Procedure 1(by Thorin Oakenshield): This is a sample stored procedure description.

Parameter 1

Parameter 2

Parameter 3

Call Procedure

[Go to homepage](#)

Figure 2: An example Stored Procedure Page

The stored procedure page includes:

- A description of the procedure functionality.

- Input boxes for all required parameters.
- A submit button that triggers the procedure using PHP and MySQL. After the
- procedure is called using the interactive button, the output of the procedure should be displayed within the page.
- A hyperlink back to the homepage.

Navigation

At the bottom of the user homepage, there is a hyperlink to the support section, where users can create and manage support tickets. This ensures smooth navigation throughout the system.

Figure 3 shows an example layout for the home page of the user.

Triggers:

Trigger 1(by Thorin Oakenshield): This is a sample trigger description.
[Go to the trigger's page](#)

Trigger 2(by Agent Smith): This is another sample trigger description.
[Go to the trigger's page](#)

Stored Procedures:

Stored Procedure 1(by Thorin Oakenshield): This is a sample stored procedure description.
[Go to the procedure's page](#)

Stored Procedure 2(by Agent Smith): This is another sample stored procedure description.
[Go to the procedure's page](#)

[Support Page](#)

Figure 3: The layout of the user's home page

2.2. Support Ticket System (User Side)

The support ticket system allows users to request help and track the status of their issues. All ticket data is stored in a MongoDB collection, and the interface is designed to be simple but functional.

Ticket List Page

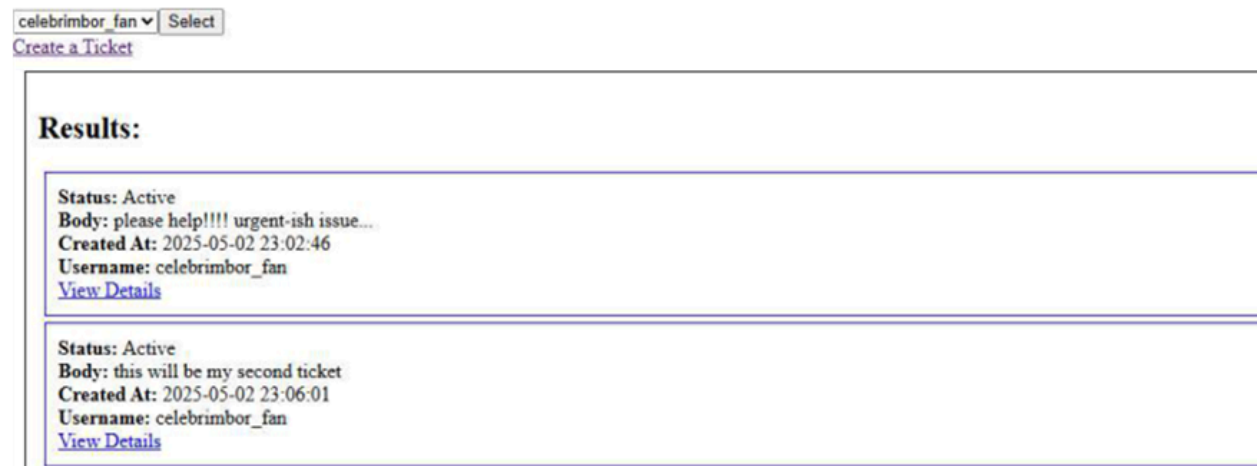
The first page of the support section presents a dropdown menu to select a username. This acts as a lightweight login system. Only users who have at least one active ticket appear in this list. Figure 4 shows how the Ticket List page should look like if there is no active ticket in the system. After selecting a username, the system displays their active tickets along with a link to create a new ticket. Figure 6 shows the usernames that have active tickets,

whereas Figure 5 shows the tickets of the username celebrimbor_fan when it is selected from the dropdown menu.



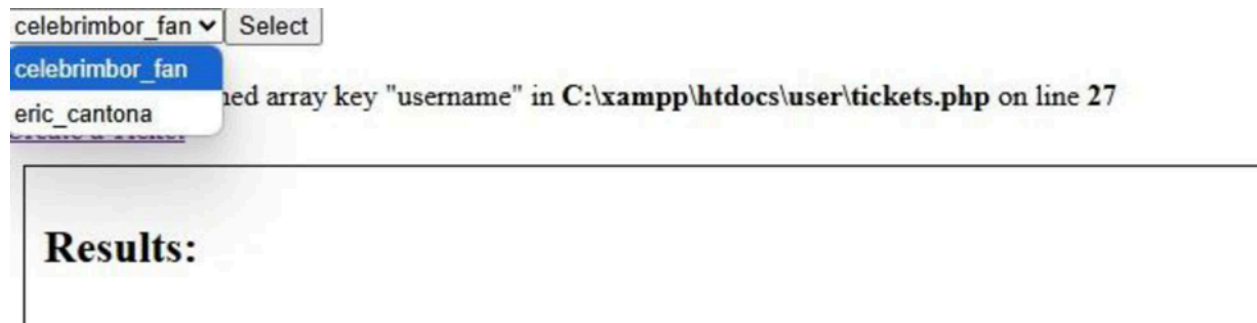
A screenshot of a web application interface. At the top, there is a dropdown menu with a downward arrow and the text 'Select', followed by a 'Create a Ticket' link. Below this, a large rectangular box contains the word 'Results:' in bold, indicating that no tickets are currently listed.

Figure 4: An example Ticket List page when there is no active tickets



A screenshot of the same web application interface, but now showing two active tickets for the user 'celebrimbor_fan'. The dropdown menu is set to 'celebrimbor_fan' and the 'Create a Ticket' link is still present. The 'Results:' section contains two ticket entries, each with the following details: Status: Active, Body: please help!!!! urgent-ish issue..., Created At: 2025-05-02 23:02:46, Username: celebrimbor_fan, and a 'View Details' link.

Figure 5: An example Ticket List page with the username filter



A screenshot of the web application interface with the dropdown menu open. The dropdown menu shows three options: 'celebrimbor_fan' (highlighted), 'eric_cantona', and 'eric_cantona'. The 'Results:' section is empty, indicating that the tickets for the selected user are not yet displayed. A message is visible in the background: 'Warning: Undefined array key "username" in C:\xampp\htdocs\user\tickets.php on line 27'.

Figure 6: Dropdown menu when there are active tickets

Ticket Creation Page

The ticket creation page includes:

- Two navigation links at the top: one to return to the homepage and another to return to the ticket list page.
- Input fields for the username and the body of the ticket message.
- A submit button to create the ticket.

Each ticket also stores additional information automatically:

- The creation date and time (as a string).
- A boolean status indicating whether the ticket is active.
- An array of comments (initially empty).

Figure 7 shows an example page layout for ticket creation.



The screenshot shows a web page for creating a ticket. At the top, there are two navigation links: [View Tickets](#) and [Home](#). Below these links is a large heading **Create a Ticket**. Under the heading, there are two input fields. The first field contains the text "celebrimbor_fan". The second field contains the text "this will be my second ticket". Below the input fields is a button labeled "Create Ticket".

Figure 7: An example Ticket Creation Page

Ticket Confirmation

After submission, the system displays a confirmation page indicating whether the ticket was successfully created. This page also includes links to both create another ticket or return to the list of tickets.

Ticket Detail View

From the ticket list, users can click on a specific ticket to view its details. The detail page shows:

- Username of the ticket creator
- Creation date
- Ticket body
- Status (active/inactive)
- Comments

Users can also add new comments to the ticket. A link is provided at the bottom to return to the ticket list page. Figure 8 shows an example layout for the ticket creation page.

Ticket Details

Username: celebrimbor_fan

Body: please help!!!! urgent-ish issue...

Status: Active

Created At: 2025-05-02 23:02:46

Comments:

[Back to Tickets](#)

Figure 8: An example ticket details page

2.3. Admin Interface

The admin interface is accessed via the localhost/admin URL and is functionally similar To the user's support system but with broader permissions.

Admin Ticket List

Unlike regular users, admins can see all active tickets from all users. The layout and features are similar, but the query in the backend retrieves a broader set of results. This page serves as the admin homepage.

Admin Ticket Detail View

Admins can view the details of any active ticket. In addition to the user's comments and metadata, admins can:

- Submit comments as the user admin (automatically assigned).
- Mark the ticket as resolved, which sets its status to false.

Once a ticket is resolved, it will no longer appear in either the user or admin homepage views, as both interfaces filter out inactive tickets. Figure 9 shows the admin's view of ticket's details.

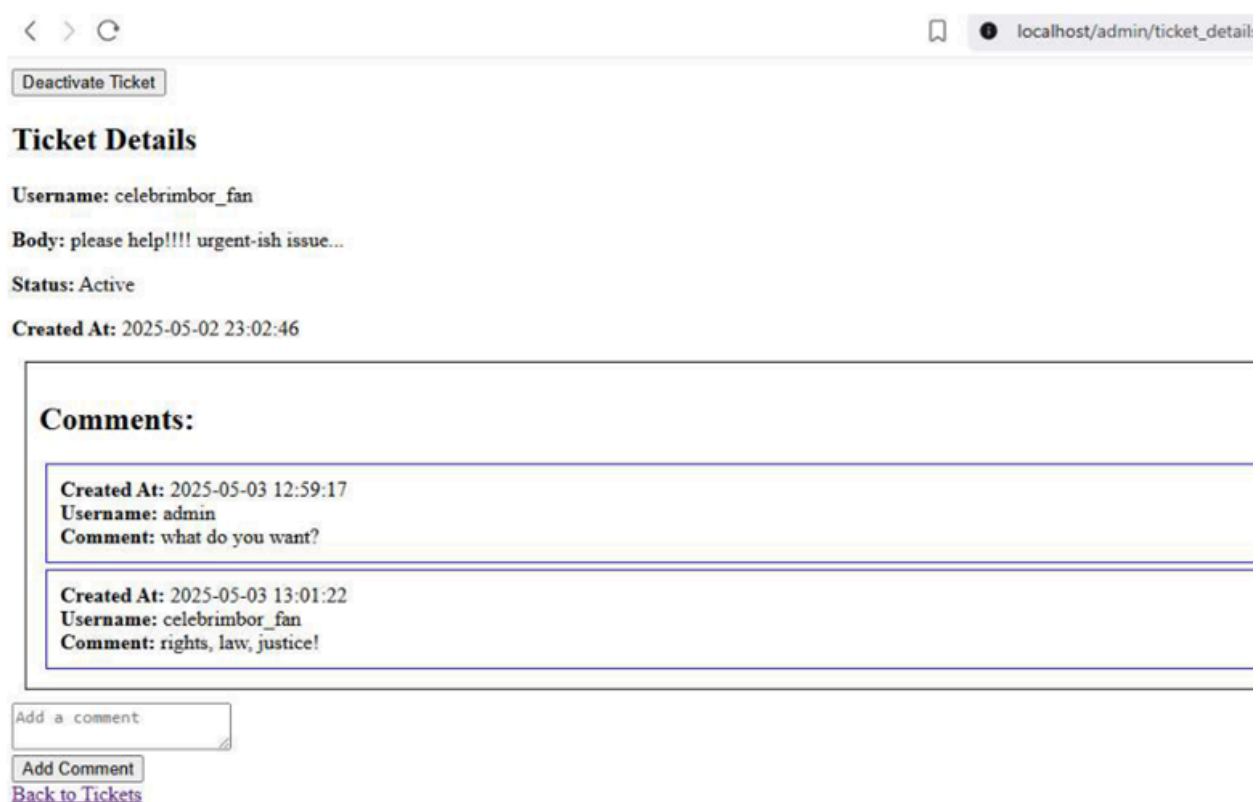


Figure 9: . Admin's view of the ticket details

3. Implementation Requirements

This section covers the technical requirements for setting up and running the project. It includes configuration steps, technology stack, folder structure, and guidelines for integrating both MySQL and MongoDB with PHP through XAMPP.

3.1. Project Structure

The project is hosted locally using the XAMPP server. The directory structure is organized under the **htdocs** folder in XAMPP as follows:

- **htdocs/user:** Contains all PHP files for the user interface, including trigger and stored procedure pages and the support ticket system.
- **htdocs/admin:** Contains the admin's interface for viewing and managing support tickets.
- Users can access the homepage via `localhost/user`, and admins can access their dashboard via `localhost/admin`.

3.2. Trigger and Stored Procedure Responsibilities

Each group member is responsible for integrating one trigger and one stored procedure. The triggers and procedures must be the ones developed in earlier phases of the database project. Integration responsibilities are displayed on the user home page next to each feature's description.

3.3. MySQL Integration

MySQL is used to manage the core relational data of the system, including the logic behind triggers and stored procedures. All database operations (e.g., calling stored procedures, activating triggers via events) are implemented using PHP's MySQLi library.

Example tasks performed using MySQL:

- Triggering actions based on database inserts/updates
- Executing stored procedures with user inputs

3.4. MongoDB Integration for Support Tickets

Support ticket operations are handled using MongoDB, with a single collection storing all ticket-related data. Each document in the collection contains:

- **username:** Creator of the ticket
- **message:** Body of the ticket
- **created at:** Timestamp (stored as a string)

- status: Boolean value (true = active, false = resolved)
- comments: Array of comment strings

Figure 10 shows the final structure of the MongoDB collections.

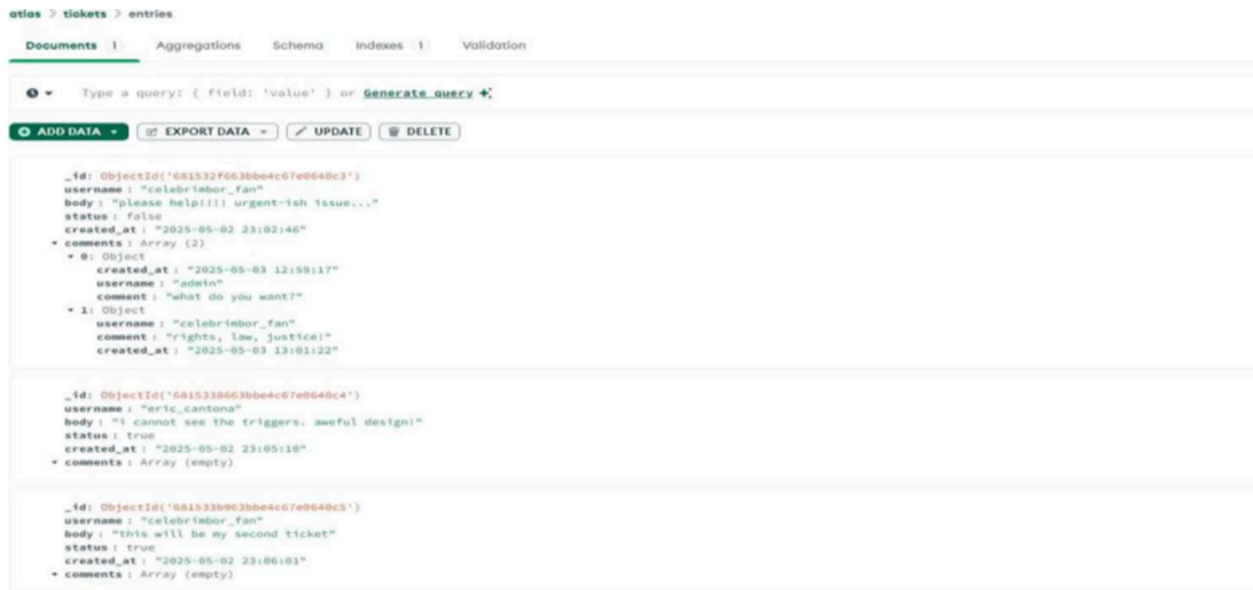
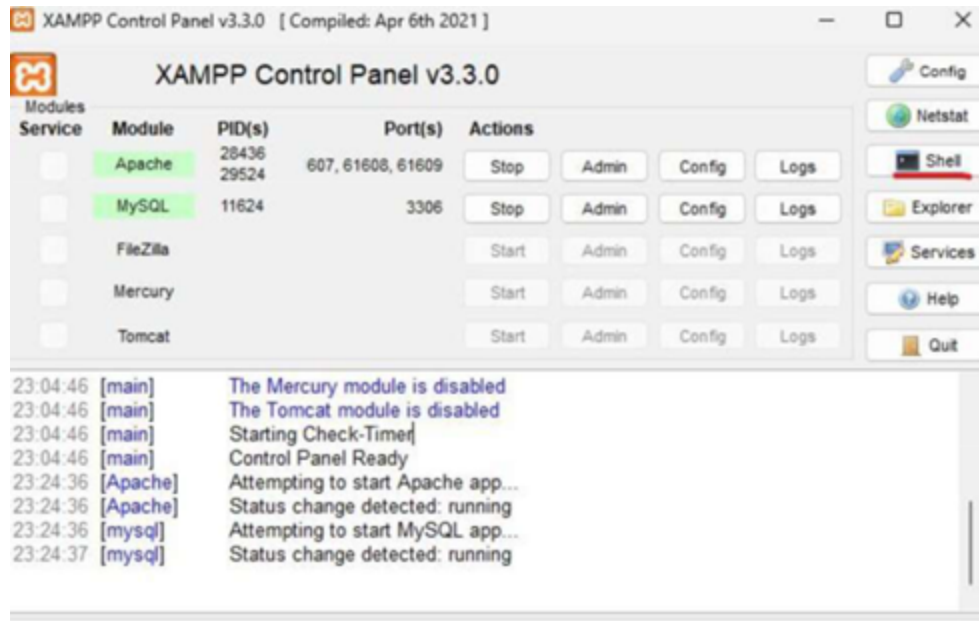


Figure 10: MongoDB Compass screenshot showing the documents in the collection

Installing the MongoDB Driver in XAMPP (Windows)

This section describes how to add MongoDB extension into PHP for Windows machines. In the next sections there will be links that may help Mac/Linux users. To enable MongoDB functionality in XAMPP on Windows:

1. Launch the terminal from the XAMPP control panel.



2. Type composer -V to check the PHP and Composer versions. The output should be similar to the following:

```
Composer version 2.8.8 2025-04-04 16:56:46
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
```

3. Visit the official PHP MongoDB driver download page to find a version matching your setup. You can find the compatible version from the following table:

Language Compatibility

The following compatibility table specifies the recommended version or versions of the PHP driver for use with a specific version of PHP.

The first column lists the driver versions.

PHP Driver Versions	PHP 8.4	PHP 8.3	PHP 8.2	PHP 8.1	PHP 8.0	PHP 7.4	PHP 7.3	PHP 7.2
ext + lib 1.21 to 2.0	✓	✓	✓	✓				
ext + lib 1.20	✓	✓	✓	✓	✓	✓		
ext + lib 1.17 to 1.19	✓	✓	✓	✓	✓	✓		
ext + lib 1.16		✓	✓	✓	✓	✓	✓	✓
ext + lib 1.15		✓	✓	✓	✓	✓	✓	✓

For more information on how to read the compatibility tables, see our guide on [MongoDB Compatibility Tables](#).

4. Download the appropriate zip file and extract the php mongodb.dll file.

5. Copy php mongodb.dll into the xampp/php/ext directory.

6. Edit php.ini (located in xampp/php) and add the following line just above

extension=curl :

extension = mongodb

7. In your project directory (inside htdocs/user and/or htdocs/admin), run:

composer require mongodb / mongodb

3.5. Focus on Functionality

While visual design is not graded, functional correctness is essential. Pages should include appropriate links for navigation, interactive elements for database integration, and meaningful status or error messages. The goal is to build a working prototype, not a polished interface.

3.6. Resources and References

The following tutorials and documentation may assist during implementation:

1. PHP + XAMPP setup: <https://www.youtube.com/watch?v=jLqBiSDNXO0>
2. PHP basics: <https://www.youtube.com/watch?v=SI69DXbYHXU>
3. GET and POST usage: <https://www.youtube.com/watch?v=6AzAYU8AOhQ>
4. PHP functions: <https://www.youtube.com/watch?v=SBFO5bIDYHQ>
5. MySQL + PHP: <https://www.youtube.com/watch?v=-1DTYAQ25bY>
6. Creating tables with phpMyAdmin: <https://www.youtube.com/watch?v=BO6XOcTyhU>
7. MySQL insertion: <https://www.youtube.com/watch?v=tq0ghtZsHJ0>
8. Executing MySQL queries in PHP: https://www.youtube.com/watch?v=om_GHI6BrdM
9. MongoDB PHP Library: <https://www.mongodb.com/docs/php-library/current/get-started/> (You are expected to use MongoDB\Driver\Manager class, do not use the code sample provided in the link)
10. MongoDB\Driver\Manager reference: <https://www.php.net/mongodb-driver-manager>
11. MongoDB Compass: <https://www.mongodb.com/products/tools/compass>
12. PHP add extension on Ubuntu: <https://stackoverflow.com/questions/78765759/add-extension-in-ubuntu-2204-to-xampp>
13. PHP add extension on Mac: <https://stackoverflow.com/questions/27886117/php-intl-installation-on-xampp>

4. Submission and Evaluation

The final submission of the project must be packaged as a single zip archive. This archive will include both the **project documentation** and the accompanying **SQL file** required to recreate the database.

4.1. Contents of the Submission

Package all files into a ZIP archive with the following structure:

```
|--- Report.pdf
|--- SQLDump.sql
|--- Scripts
|   |-- user
|   |   |-- index.php
|   |   |-- script_1.php
|   |   |-- .
|   |   |-- script_n.php
|   |-- admin
|   |   |-- index.php
|   |   |-- script_1.php
|   |   |-- .
|   |   |-- script_m.php
```

Name your ZIP file as **GroupXX_phase4.zip** and submit into Sucourse

PDF Report:

A comprehensive document detailing the structure and functionality of the system.

The FIRST PAGE of the PDF report must include the course name, project title (Phase 4), group number, and the names of ALL group members (with roll numbers).

1. For each trigger;
 - a. provide the SQL script that will create it,
 - b. state the different cases and corresponding buttons in the web page of it.
2. For each stored procedure;
 - a. provide the SQL script that will create it
 - b. state the input parameters and corresponding input boxes in the web page of it.
3. For each MongoDB query implemented for the support system;
 - a. provide the PHP script that corresponds to the query.

SQL File:

A single SQL script file containing the statements for:

- Creation of the database and relevant tables.
- Insertion of mock data to test the project.
- Definitions of all triggers and stored procedures used in the project.

Scripts:

All the PHP scripts of the web application. script i.php is a placeholder, you can name your scripts freely.

4.2. Demo and Grading Policy

Project evaluation will be based on both the submitted PDF report and the live demo with TAs.

4.2.1. Demo (70%)

Each group must demo in person/Zoom , with a **maximum length of 10 minutes**, demonstrating:

- The correct execution of each trigger and stored procedure
- The full workflow of the support ticket system (ticket creation, listing, management)
- Speaker could be one of the group members and clearly speak about what he/she does during demo.
- Show the actual environment, including the running servers, database connections, and web interfaces (MongoDB Compass, XAMP control panel, WebBrowser, MySQL and etc).

Before starting Demo, each group must ensure that the following software and configurations are correctly installed and **visible**:

- XAMPP server with Apache and MySQL running
- MongoDB Compass (or any interface used to inspect the MongoDB ticket database)
- Configured MongoDB PHP extension and functional project folder
- Browsers or tools used to display web interfaces
- The demo must clearly demonstrate:
 - Working web interfaces connected to each trigger and stored procedure Functional ticket creation, listing, and management
- Although the support ticket system is developed collaboratively, the speaker must be able to present and run it.

4.2.2. PDF Report (20%):

Students must submit a well-structured PDF report containing:

A brief explanation of each implemented trigger and stored procedure
Screenshots of successful executions

A description of the support ticket system workflow
Any assumptions, challenges, or design decisions
The final SQL file and PHP integration notes (as appendices if needed)

4.2.3. Scripts and SQL Files (10%)

Include all scripts and SQL code to your projects. They should be executable with no syntax errors.